



دانشکده علوم ریاضی
دانشگاه فردوسی مشهد

پروژه درس برنامه سازی پیشرفته

بازی چهار مهره در یک ردیف

استاد درس: دکتر محمود امین طوسی

اعضای گروه:

اسما موحدی تبار

فهرست مطالب

۲	۱	مقدمه
۲	۲	توضیح کلی کد
۲	۱.۲	Board class
۲	۲.۲	Player class
۲	۳.۲	AI class
۳	۴.۲	Game class
۴	۳	توضیح جزئی کد
۴	۱.۳	Board class
۴	۱.۱.۳	printBoard()
۴	۲.۱.۳	isValidMove
۴	۳.۱.۳	makeMove
۴	۴.۱.۳	numberOf
۵	۵.۱.۳	checkFor
۵	۲.۳	Player class
۵	۳.۳	AI class
۵	۱.۳.۳	getMove
۶	۴.۳	Game class
۶	۱.۴.۳	selectMode
۶	۲.۴.۳	play
۶	۳.۴.۳	main() int

۱ مقدمه

در این پروژه بازی چهار مهره در یک ردیف به روش شیء گرا پیاده سازی شده است.

۲ توضیح کلی کد

این کد دارای ۴ کلاس، board، ai player، game و می باشد.

۱.۲ Board class

کلاس board که زمینه یا محیط بازی را میسازد دارای اعضای تعداد ردیف ها (ROWS=۶) و تعداد ستون ها (COLS=۷) و وکتوری دو بعدی با نام شبکه (که دارای بعدهای ستون و ردیف است). است که جایگاه مهره ها را تشکیل میدهد. این کلاس دارای تابع هایی با نام های printBoard که برد را چاپ میکند؛ isValidMove که مجاز بودن حرکت خواسته شده را بررسی میکند؛ makeMove که اعمال حرکت خواسته شده برای طرف خواسته شده را روی صفحه بازی انجام میدهد؛ numberOf که وظیفه اش شمردن تعداد مهره های ۴ تایی کنار هم برای بازیکن خواسته شده در انتهای بازی است و checkFor3 که چک میکند که آیا با حرکت داده شده ۳ یا ۴ مهره کنار هم با نماد خواسته شده یکسان تشکیل میشود یا نه است. این ها همه کارهایی است که مربوط به صفحه بازی است، در نتیجه در این کلاس قرار گرفته است.

۲.۲ Player class

اعضای خصوصی این کلاس نام و نماد (شناسای مالکیت مهره است و معمولاً X/O است) است. عمومی: تابع سازنده که با گرفتن نام و نماد یک بازیکن با آن نام و نماد میسازد. به علاوه این کلاس دارای یک تابع getter برای نام است.

۳.۲ AI class

این کلاس نقش رفتار هوش مصنوعی را دارد که هنگام بازی با کامپیوتر نیاز میشود و تنها دارای تابع getMove است. این تابع صفحه و نماد کامپیوتر و نماد حریف را میگیرد تا در هر نوبت با کمک تابع checkFor3 بررسی کند کدام حرکت از کل ستون ها بیشتر به نفع کامپیوتر است و آن حرکت را خروجی دهد و اگر حرکتی به نفع تر وجود نداشت با استفاده از احتمال توزیع گاوسی (چون هرچه وسط تر باشد فرصت تشکیل ۴ تایی بیشتر است).

حرکت را انتخاب کند.

Game class ۴.۲

این کلاس نقش گرداننده بازی را به عهده دارد و دارای دو تابع `play` و `selectMode` است. که اولی به بازیکن امکان انتخاب حالت بازی (دو نفره یا تک نفره که نفر مقابل کامپیوتره) را میدهد و دومی: به اندازه ۴۲ بار (تعداد سلول ها) نماد نوبت را مشخص میکند؛ حرکت را از بازیکن (انسان/کامپیوتر) میگیرد و مجاز بودنش را بررسی میکند و اگر مجاز بود حرکت را روی صفحه بازی انجام میدهد وگرنه خطا میدهد و نوبت بازیکن را حفظ میکند تا دوباره انتخاب کند. در نهایت که حلقه تمام شد تعداد ۴ تایی هر بازیکن را با تابع مربوط در کلاس `Board` مشخص میکند و در نتیجه برنده مشخص میشود. تابع `main` به طور مستقیم فقط از این کلاس استفاده میکند.

۳ توضیح جزئی کد

Board class ۱.۳

اعضای خصوصی این کلاس همانطور که گفته شد ستون و ردیف و شبکه هستند که شبکه یک و کتر دو بعدی با اعضای کاراکتری است که در ابتدا همه اعضا برابر ” هستند و به مرور هر کدام شامل نمادی از مهره های بازیکن ها میشوند. پس در ابتدای بازی در تابع سازنده برد شبکه دو بعدی ای با اندازه بعد های ROWS=۶ و COLS=۷ و محتویات کاراکتری ’ ’ تشکیل میدهیم. تابع ها:

printBoard() ۱.۱.۳

این تابع با دو حلقه for تودرتو در هر ردیف، محتویات هر ستون آن ردیف را بین دو | | چاپ میکند و بعد به ردیف بعد می رود تا انتها و از لحاظ مختصات هر سلول مطابق دستگاه مختصات (- وای، + ایکس) عمل میکند. در انتها با خط چین مرزی میکشد و شماره ستون ها را برای انتخاب کاربر چاپ میکند. این تابع خروجی ای ندارد. (void)

isValidMove ۲.۱.۳

این تابع عدد ستون انتخابی کاربر را به عنوان آرگومان دریافت میکند و مجاز بودن ستون را از لحاظ بودن در بازه تعداد ستون هایی که صفحه بازی دارا میباشد و همچنین پر نبودن آن ستون (== grid[۰][col]) بررسی میکند. اگر همه این شرایط مهیا بود true و اگر هر کدام اشتباه باشد false را برمیگرداند.

makeMove ۳.۱.۳

این تابع با گرفتن آرگومان های ستون و نماد در ستون مورد نظر از پایین به بالا ردیف ها را چک میکند و در اولین ردیفی که نمادی نباشد (== ') نماد داده شده را قرار میدهد و از حلقه ردیف ها بیرون می آید. خروجی این تابع تهی است. (void)

numberOff ۴.۱.۳

این تابع هم با گرفتن نماد بازیکن تعداد ۴ تایی های پیوسته ای که آن بازیکن در حالت های مختلف (افقی - عمودی - اریب) ساخته است را می شمرد و در s که در ابتدا ۰ تعریف شده ذخیره میکند و در نهایت مقدار عددی s را برمیگرداند. در این شبکه آرایه ی اول متعلق به ردیف و دومی متعلق به ستون است. (۱) حالت افقی: برای هر ردیف از ستون اول تا سه تا مونده به آخر را تک به تک چک میکند که آیا آن ستون تا ۳ ستون بعدش به طور پیوسته دارای

نماد موردنظر میباشند یا نه. اگر بودند به s یکی اضافه میکند وگرنه هیچی. (۲) حالت عمودی: برای هر ردیف از ردیف اول تا ۳ تامونده به آخر تمام ستون ها را چک میکند که اگر در ستونی واحد از ردیف موردنظر تا ۳ تا بعدترش به طور پیوسته دارای نماد موردنظر بودند s را یکی بیشتر کند. (۳) حالت اریب از بالای چپ به پایین راست: برای هر ردیف و ستون از اول تا ۳ تامونده به آخر را چک میکند که اگر در سلولی واحد تا ۳ ستون و ۳ ردیف بعدش به طور پیوسته دارای مهره با نماد موردنظر هستند s یکی بیشتر میشود وگرنه هیچی. (۴) حالت اریب از بالای راست به پایین چپ: همان مورد ۳ ولی برعکس.

۵.۱.۳ checkFor3

این تابع با گرفتن نماد و ستون مختصات مهره با نماد موردنظر را به دست آورده سپس از آن نقطه تا ۳ و ۴ تا سلول بعدترش در همه جهت ها چک میکند و می‌شمرد. اگر در هر جهت ۴ جایگاه پیوسته دارای آن نماد یکسان باشند بلافاصله ۴ را برمیگرداند و اگر ۳ جایگاه دارای آن نماد شوند ۳ را در output ذخیره میکند تا همه جهت ها تمام شوند و مطمئن شود به ۴ تا نمیرسد؛ در نهایت مقدار output را برمیگرداند. شیوه چک کردن هم به همان روش numberOf4 است.

۲.۳ Player class

جزئیات این کلاس برابر همان کلیاتش است.

۳.۳ AI class

۱.۳.۳ getMove

در این تابع که برای پیدا کردن مناسبترین حرکت ممکن کامپیوتر نوشته شده board و نماد کامپیوتر و حریف داده میشود. در این تابع تمام ستون ها برای ۴ تایی شدن کامپیوتر یا حریف چک میشود و اگر چنین ستونی وجود داشت بلافاصله شماره آن ستون به عنوان بهترین انتخاب حرکت خروجی داده میشود. ولی اگر ستونی پیدا شود که باعث ۳ تایی شدن کامپیوتر شود در col3 ذخیره میشود تا همه ستون ها چک شود که در صورت نبودن گزینه بهتر، آن برگردانده شود. و اگر، نه ۳ تایی و نه ۴ تایی ای پیدا شد عددی رندوم با توزیع گاوسی برگردانده میشود. به این صورت که شانس ستون ها از وسط به کنار کاهش پیدا میکند. برای این کار آرایه ای نوشتم تا گزینه های موجود با ضریب دلخواه را داخلش بنویسم و از بین اعضای این آرایه با rand() عددی رندوم انتخاب شود که شماره خانه یکی از اعضاست. و به این وسیله شماره ستون تصادفی در صورت مجاز بودن برگردانده میشود.

Game class ۴.۳

کلاس گیم جایی است که همانطور که در اعضای خصوصیش میبینید از همه کلاس های دیگه استفاده میشه و نقش اصلی یا گرداننده بازی را دارد. در این کلاس یک صفحه، دو بازیکن، یک هوش مصنوعی میسازیم؛ به علاوه دو عضو دیگه که یکی برای تشخیص بازی با کامپیوتر است و دیگری برای نماد در هر نوبت. تابع سازنده گیم به طور پیشفرض دو بازیکن با نماد های O ، X و نام های Player۱ ، Player۲ میسازد. به علاوه به طور پیشفرض دو بازیکن را انسان در نظر میگیرد و نوبت اول را به X/Player۱ میدهد.

selectMode ۱.۴.۳

این تابع مقدار vscomputer را بر اساس خواست بازیکن تنظیم میکند. پس یک انتخاب از کاربر ورودی میگیرد که اگر ۱ باشد یعنی دو انسان و اگر ۲ باشد یعنی کامپیوتر و انسان با هم بازی میکنند. خروجی این تابع تهی است (void).

play ۲.۴.۳

این تابع در ابتدا مقداردهی اولیه به تابع تولید اعداد تصادفی بر حسب زمان میکند که باعث میشود مقدار rand() هر دفعه متفاوت شود. سپس حلقه ای تشکیل میدهد که تا ۴۲ بار (تعداد سلول های خالی بازی) اعمال داخلش انجام شود: در ابتدا صفحه بازی چاپ میشود تا وضعیت را نشان دهد. سپس بر اساس زوج یا فرد بودن شمارنده حلقه نمادی که در هر نوبت باید حرکت انجام دهد مشخص میشود. اگر کاربر درخواست بازی با کامپیوتر را داشته نماد O مربوط به کامپیوتر است و هروقت هم نوبت O بود هم vsComputer = true یعنی نوبت کامپیوتر است. اگر نوبت کامپیوتر بود که با تابع getMove هوش مصنوعی حرکت انتخاب میشود و چون در نمایش به جای * از ۱ شروع کرده ایم برای چاپ حرکت یکی به آن اضافه میشود و چاپ میشود. و در غیر این صورت یعنی نوبت بازیکن انسان است که حرکت از او گرفته میشود و به همان دلیل شروع از ۱ یکی از آن کم میشود. سپس مجاز بودن آن چک میشود و اگر مجاز بود حرکت انجام میشود و اگر نبود خطا داده میشود و نوبت با اضافه نشدن به شمارنده حلقه حفظ میشود تا دوباره بازیکن انتخاب کند. هنگامی که حلقه تمام میشود یعنی بازی تمام شده و با numberOff۴ کلاس board تعداد ۴ تایی هر بازیکن مشخص میشود و پیغام مناسب وضعیت اتمام بازی داده میشود. خروجی این تابع تهی است (void).

main() int ۳.۴.۳

در این بخش فقط یک کلاس گیم میسازیم و با تابع های این کلاس بازی انجام میشود.